

Writeup: – PasswdGuessr

Informations sur le Crackme

- **Équipe cible** : Non spécifiée
- **Nom du fichier** : writeup12_passwdgsrrOK.txt
- **Difficulté estimée** : Non spécifiée
- **Flag découvert** : `A1b2C3d4E5f6G7h8`

Résumé

Objectif : récupérer le flag de 16 caractères que le programme attend.

1. Identification du binaire

```
file PasswdGuessr
```

Outils Utilisés

Décodage de la table

```
python3 - <<'PY'  
table = bytes.fromhex("1b6b386819693e6e1f6f3c6c1d6d3262")  
flag = bytes(b ^ 0x5A for b in table)  
print(flag.decode())
```

Outils utilisés : file, readelf, hexdump, objdump,script Python.

Analyse Statique

Carto des sections ELF

```
$ readelf -S PasswdGuessr  
.text addr 0x401000 off 0x1000 size 0xC6  
.data addr 0x402000 off 0x2000 size 0x52  
.bss addr 0x402054 off 0x2054 size 0x24
```

J'ai noté que la section .data fait 0x52 octets à l'offset 0x2000.

Inspection de la section .data

```
hexdump -C -s 0x2000 -n 0x52 PasswdGuessr
```

```
00002000 45 6e 74 72 65 7a 20 6c 65 20 6d 6f 74 20 64 65 |Entrez le mot  
de|  
00002010 20 70 61 73 73 65 20 3a 20 00 41 63 63 c3 a8 73 | passe : .Acc..s
```

```
|
00002020 20 61 75 74 6f 72 69 73 c3 a9 20 21 0a 00 41 63 | autoris.. !..Ac|
00002030 63 c3 a8 73 20 72 65 66 75 73 c3 a9 2e 2e 2e 0a |c..s refus.....|
00002040 00 1b 6b 38 68 19 69 3e 6e 1f 6f 3c 6c 1d 6d 32 |..k8h.i>n.o<l.m2
|
00002050 62 5a                                     |bZ|
```

Après le byte 00 se trouvent 17 octets suspects (1b 6b ... 62 5a)

Le dernier (5a) ressemble à une sentinelle.

J'ai isolé les 16 octets chiffrés :

1b 6b 38 68 19 69 3e 6e 1f 6f 3c 6c 1d 6d 32 62

Analyse de l'assembleur

```
objdump -d -M intel PasswdGuessr | less
```

je suis directement allée à la boucle de comparaison (recherche cmp bl,al) :

```
_start.compare:
xor    rsi, rsi
_loop:
mov     al, BYTE PTR [rsi+0x402041]
xor     al, 0x5a
mov     bl, BYTE PTR [rsi+0x402054]
cmp     bl, al
jne     _start.fail
cmp     al, 0
je      _start.success
inc     rsi
jmp     _loop
```

J'ai compris que le programme

lit table[i],

fait XOR 0x5A,

compare au caractère saisi.

Dès qu'un AL final vaut 0 → mot de passe terminé.

Analyse Dynamique

Je le rends exécutable :

chmod +x PasswdGuessr

Lancer une première fois pour voir l'I/O

```
./PasswdGuessr
```

Identification du Mécanisme de Validation

Entrez le mot de passe :....

Accès refusé...

J'ai constaté qu'il lit le mot de passe sur stdin puis répond.

Validation

```
echo -n "A1b2C3d4E5f6G7h8" | ./PasswdGuessr
```

Entrez le mot de passe : Accès autorisé !

J'ai bien le message de succès.

Découverte du Flag

Donc :

`password[i] = table[i] XOR 0x5A`

Le flag est la chaîne : A1b2C3d4E5f6G7h8

Conclusion

Conclusion

Protection : simple XOR byte-à-byte avec une constante (0x5A).

Méthode : reverse manuel.

PasswdGuessr

PasswdGuessr: ELF 64-bit LSB executable, x86-64,...

J'ai vérifié qu'il s'agit d'un exécutable x86-64 ELF.

PY

PY

J'ai obtenu le mot de passe clair : A1b2C3d4E5f6G7h8